Lecture 11

Scripting and Automation.

Accessing the Short Read Archive

Scripts

Collect multiple commands into a single text file.

Run the same commands again.

Also used document the steps and describe the thought process!

Think of it as a **Lab Book** for analysis.

Who do you write this document for?

The Future You.

Today everything seems crystal clear.

Next month it won't be so.

Good documentation is the ART of predicting what you and others won't understand about the process.

How much to document?

Making it verbose is a lot of work. Keepin it up to date challenging. Misleading documentation is sometimes worse than no documentation.

But having too little means you won't understand what you did.

Don't write not too much. Don't write too little 👀

Write simple, brief, factual, short senteces.

Documenting is a skill

Make a note of what typicaly confuses you when you run an analysis later.

Make a note of documentation that is redundant.

What is better?

```
# I am running trimmommatic for QC
trimmomatic SE input.fq output.fq SLIDINGWINDOW:4:30
```

```
# Trim back reads by quality.
trimmomatic SE input.fq output.fq SLIDINGWINDOW:4:30
```

You need a REAL text editor

Microsoft Word is **NOT** a text editor.

Required features

- Your text should have line numbering.
- You should be able to visualize **whitespace** (`SPACES` vs `TABS`)

IT is also important that you should be able to switch `NEW LINES` Windows vs Unix

Recommended Editors

- NotePad++ on Windows
- Sublime Text 2 available for all platforms
- Komodo Edit works on all platforms

It can be suprisingly difficult to make modern editors use the `TAB` character even when you need that.

Typicall, by default, editors insert 4 `SPACE` characters when you press `TAB`. Can be very confusing!

There is a setting that you need to override. (Google)

Collect your commands into a file

```
1
2  # Comments start with the # sign.
3
4  # Work at the command line.
5  # If it works copy it into this file.
6
7
8  # Get the example dataset for lecture 11.
9  wget http://data.biostarhandbook.com/data/sequencing-platform-data.tar.gz
10
11 # The file unpacks into 4 sequencing datasets.
12 tar zxvf sequencing-platform-data.tar.gz
13
14 # Quality plots before trimming.
15 fastqc illumina.fq
16
17 # Trim back by quality.
18 trimmomatic SE illumina.fq better.fq SLIDINGWINDOW:4:30
19
20 # Quality plots after trimming.
21 fastqc better.fq
22
```

Execute the script

If you called your file `lecture11.sh` then you can run it with:

```
bash lecture11.sh
```

As you work on your pipeline you can temporarily "comment out" the previous steps so you don't have to wait on processes that you know that work.

Now you have written your first script.

Code Refactoring.

The ART of making reusable components.

Refactoring

Separate the **changing** from the **non-changing** parts.

Identify what changes and what stays the same between the two lines:

```
fastqc illumina.fq
trimmomatic SE illumina.fq better.fq SLIDINGWINDOW:4:30
```

```
fastqc iontorrent.fq
trimmomatic SE iontorrent.fq better.fq SLIDINGWINDOW:4:30
```

Replace changing parts with **variables**.

Using variables

Assign the changing section to a variable (`$`).

```
DATA=illumina.fq
```

Use the variable.

```
fastqc $DATA
trimmomatic SE $DATA better.fq SLIDINGWINDOW:4:30
```

You can now change the variable and you won't need to change the actions:

```
DATA=iontorrent.fq
```

Refactor our script

Move all variable content to the top:

```
# The original input data.
DATA=illumina.fq

# The improved data.
TRIMMED=better.fq

# ----- No changes required below. -----

# Quality plots before trimming.
fastqc $DATA

# Trim back by quality.
trimmomatic SE $DATA $TRIMMED SLIDINGWINDOW:4:30

# Quality plots after trimming.
fastqc $TRIMMED
```

Refactoring is an iterative process.

Make your code "readable" by self documenting it.

Move variables out of the script

Even better way would be to move the variable content all the way out of the script.

So you could run it this way:

```
bash lecture11.sh illumina.fq
```

and this way:

```
bash lecture11.sh iontorrent.fq
```

Now you don't even need to edit the code (and potentialy make a mistake).

The command line parameters

`$1`, `$2`, `$3` are special variables that come from "outside" the script.

```
echo "Hi: $1!"
echo "Bye: $2!"
```

Run it with:

```
bash sayhello.sh Jane Joe
```

prints:

```
Hi: Jane!
Bye: Joe!
```

Our generic trimmer script

```
# The original input data.
DATA=$1

# The improved data.
TRIMMED=better.fq

# Quality plots before trimming.
fastqc $DATA

# Trim back by quality.
trimmomatic SE $DATA $TRIMMED SLIDINGWINDOW:4:30

# Quality plots after trimming.
fastqc $TRIMMED
```

Shell variable naming

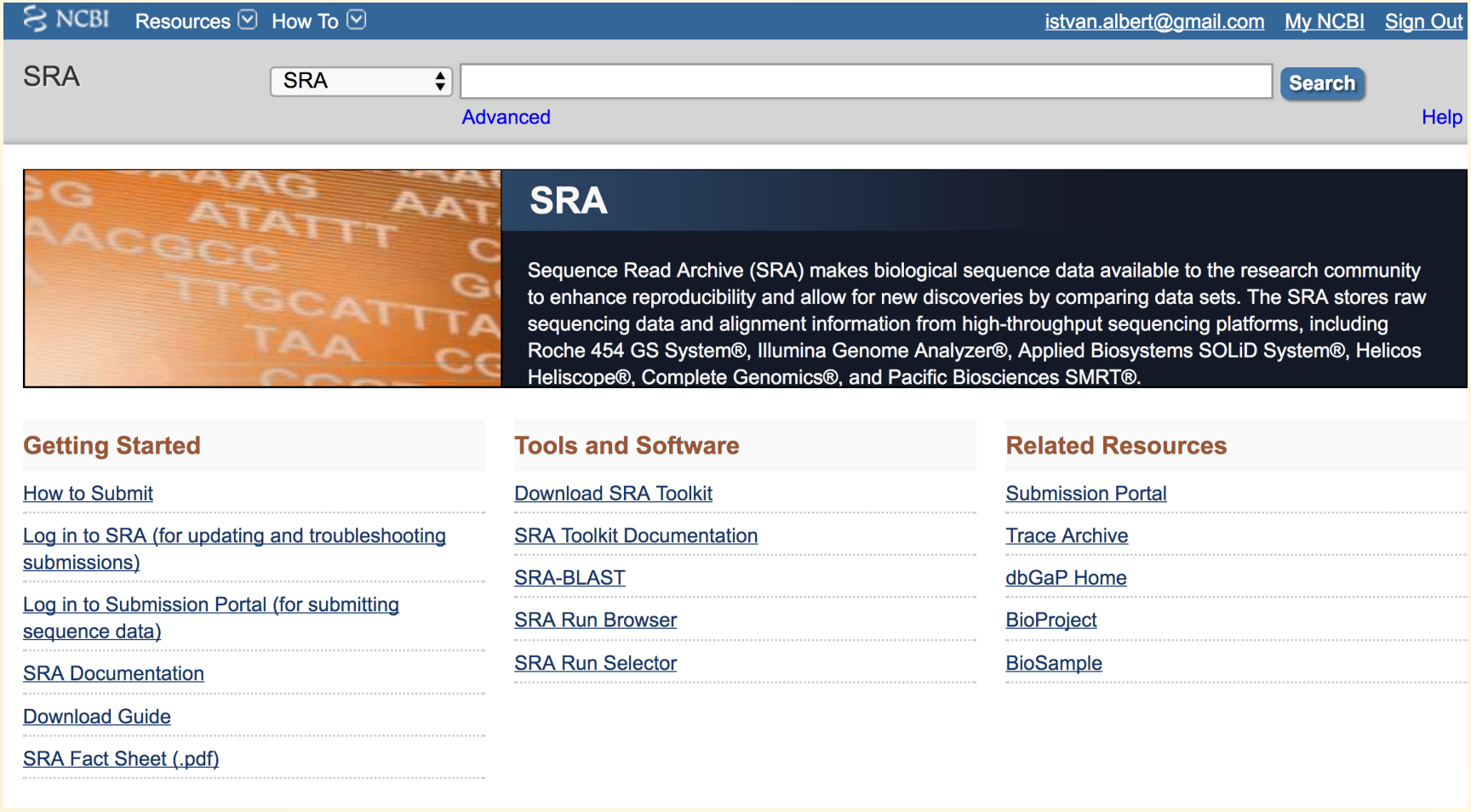
Practice shell variable by typing into your shell. Bash supports both a *short form* `$` and a *long form* `${}` variable access:

```
A=FOO
```

Predict what each will print:

```
echo A
echo $A
echo ${A}
echo $ABAR
echo ${A}BAR
```

The Short Read Archive



Data is deposite under run numbers like `SRR5119926`,...

The SRA toolkit

We will cover later how to search for SRA accession numbers.

If you knew the accession number the `fastq-dump` command can download the data for it:

```
fastq-dump -X 15000 --split-files SRR5119926
```

The `-X` will extract only a subset (1500 reads).

Note: Some datasets may be very large. Alas `fastq-dump` will first download the entire data even if you only need a small section of it ... sigh!

Automating SRA access

Let's write a script that operates at the command line like so:

```
bash getdata.sh SRR5119926
```

Downloads a subset of the data for run `SRR5119926` then generates quality control plots for it.

Build it one step at a time.

Use the `echo` command to print variables.

The SRA data download script

Add a brief intro with usage example to your scripts:

```
#
# This program downloads and unpacks FASTQ files corresponding
# to an SRR run then performs trimming on it.
#
# Usage: bash getdata.sh SRR5119926
#

SRR=$1

echo "Run id: ${SRR}"

# Limit to a subset for now.
fastq-dump -X 1500  --split-files ${SRR}

# Paired end data is named by _1 and _2.
fastqc ${SRR}_1.fastq ${SRR}_2.fastq

```